Lab 2: Install and Run Kafka in KRaft Mode

Objectives

- Install Apache Kafka on a Linux machine using the same binary distribution.
- Configure and format a single-node KRaft cluster (combined broker and controller).
- Start Kafka in KRaft mode and verify basic messaging.

Lab Topology

- One Linux VM or server.
- One Kafka process running as both broker and controller.
- PLAINTEXT client listener on `localhost:9092` and controller listener on `localhost:9093`.

Step 1: Reuse or Install Kafka

If Lab 1 was completed on the same machine, reuse the same `KAFKA_HOME` directory. Otherwise, repeat the Java installation and Kafka download/extract steps described previously.

Recent Kafka distributions include a `config/kraft` directory with example KRaft configuration files such as `server.properties` tailored for KRaft mode. Commands in this lab assume the presence of `config/kraft/server.properties` as provided by upstream Kafka releases.

Step 2: Configure KRaft Server Properties (Single Node)

1. Open `config/kraft/server.properties` in an editor.
2. For a combined broker/controller on a single node, ensure the following properties are set (actual file may already contain similar defaults):
    - `process.roles=broker,controller`.
    - `node.id=1`.
    - `controller.quorum.voters=1@localhost:9093` (single voter, same node).
    - `listeners=PLAINTEXT://:9092,CONTROLLER://:9093`.
    - `controller.listener.names=CONTROLLER`.
    - `listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT`.
    - `log.dirs=/tmp/kraft-combined-logs`.
3. Save the file.

The `process.roles` and `controller.quorum.voters` properties are central to KRaft's metadata quorum configuration. Single-node setups typically collocate broker and controller for simplicity, but multi-node deployments separate or replicate controllers for higher availability.

Step 3: Generate a Cluster ID

1. From `KAFKA_HOME`, run:
    - `KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"`.

2. This command stores a newly generated UUID in the `KAFKA_CLUSTER_ID` environment variable that will be used to format the KRaft storage.

Kafka documentation prescribes using `kafka-storage.sh random-uuid` to generate a cluster ID before formatting KRaft storage directories.

Step 4: Format KRaft Storage Directories

1. Still in `KAFKA_HOME`, format the log directory configured in `log.dirs`:
    - `bin/kafka-storage.sh format --standalone -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties`.
2. Successful output should mention formatting the specified log directory, for example `/tmp/kraft-combined-logs`.

The `format` step writes a `meta.properties` file containing the cluster ID and other metadata; Kafka will not start in KRaft mode if the directory is not formatted.

Step 5: Start the KRaft Broker/Controller

1. Start the Kafka server in KRaft mode:
    - `bin/kafka-server-start.sh config/kraft/server.properties`.
2. Confirm in the logs that the node starts as both broker and controller and that the controller quorum is established for the single voter configuration.

At this point, a single-node KRaft-based Kafka cluster is running and listening for client connections on port 9092.

Step 6: Create and Inspect a Topic

1. In a new terminal, from `KAFKA_HOME`:
    - `bin/kafka-topics.sh --create --topic kraft-test --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1`.
2. List topics:
    - `bin/kafka-topics.sh --list --bootstrap-server localhost:9092`.
3. Describe the topic:
    - `bin/kafka-topics.sh --describe --topic kraft-test --bootstrap-server localhost:9092`.

The topic administration commands are identical to those used in ZooKeeper mode, differing only in which cluster and metadata backend they talk to.

Step 7: Produce and Consume Messages

1. Start a console consumer:
    - `bin/kafka-console-consumer.sh --topic kraft-test --from-beginning --bootstrap-server localhost:9092`.
2. Start a console producer in another terminal:
    - `bin/kafka-console-producer.sh --topic kraft-test --bootstrap-server localhost:9092`.
3. Send sample messages from the producer and observe them in the consumer to confirm end-to-end flow.

As with ZooKeeper mode, this simple loopback test validates the correctness of installation and configuration in KRaft mode.

Step 8: Stop KRaft Node and Reset

1. Stop producer and consumer with `Ctrl+C`.
2. Stop the KRaft node with `Ctrl+C` in the server terminal.
3. To reset the environment fully, delete the log directory configured in `log.dirs` (for example, `/tmp/kraft-combined-logs`) and rerun the `kafka-storage.sh format` step with a new or the same cluster ID before restarting.

Deleting the log directory removes `meta.properties` and topic data, so Kafka must be reformatted before it can start again in KRaft mode.

Multi-Controller, Multi-Broker KRaft Cluster on One Machine

Objectives (Extension)

- Configure three dedicated KRaft controllers on a single Linux VM.
- Configure three dedicated brokers on the same VM, each using a different port and log directory.
- Form a single KRaft cluster with 3 controllers and 3 brokers and verify a replicated topic.

Extended Lab Topology

- One Linux VM or server.
- Three controller processes (controller-only nodes).
- Three broker processes (broker-only nodes).
- All processes share the same CLUSTER_ID but use unique node.id, ports, and log.dirs.kafka.apache+2

Step 9: Prepare Separate Config Files

From KAFKA_HOME, create separate configuration files for each controller and broker:

bash

```
cd $KAFKA_HOME

# Controllers
cp config/kraft/server.properties config/kraft/controller-1.properties
cp config/kraft/server.properties config/kraft/controller-2.properties
cp config/kraft/server.properties config/kraft/controller-3.properties

# Brokers
cp config/kraft/server.properties config/kraft/broker-1.properties
cp config/kraft/server.properties config/kraft/broker-2.properties
cp config/kraft/server.properties config/kraft/broker-3.properties
```

You will edit these six files to assign unique roles, node IDs, ports, and log directories.kafka.apache+1

Step 10: Configure Three Controller-Only Nodes

Edit each controller-*.properties file so that it runs as a controller only, with a unique node.id, controller port, and log directory.

1. Open config/kraft/controller-1.properties and set (adjust any existing values):

```
process.roles=controller
node.id=1

# Controller listener (first controller)
listeners=CONTROLLER://:19093
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT

# All controllers in the quorum (same on every node)
controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093

# Log directory for this controller
log.dirs=/tmp/kraft-controller-1-logs
```

2. Open config/kraft/controller-2.properties:

```
process.roles=controller
node.id=2

listeners=CONTROLLER://:29093
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT

controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093

log.dirs=/tmp/kraft-controller-2-logs
```

3. Open config/kraft/controller-3.properties:

```
process.roles=controller
node.id=3

listeners=CONTROLLER://:39093
controller.listener.names=CONTROLLER
```

listener.security.protocol.map=CONTROLLER:PLAINTEXT

controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093

log.dirs=/tmp/kraft-controller-3-logs

Key points:

- process.roles is controller on these nodes (they do not accept client traffic).kafka.apache+1
- Each controller has a unique node.id (1, 2, 3) and its own controller port and log.dirs.kafka.apache+1
- controller.quorum.voters lists all controllers and uses the same value in all six configs (controllers and brokers).github+2

Step 11: Configure Three Broker-Only Nodes

Now configure brokers that talk to the controller quorum. Each broker has a **unique** node.id that does **not** appear in controller.quorum.voters, plus a unique client port and log directory.books.japila+1

1. Open config/kraft/broker-1.properties:

process.roles=broker

node.id=4

Client listener for this broker

listeners=PLAINTEXT://:9092

advertised.listeners=PLAINTEXT://localhost:9092

Map both client and controller listener names to PLAINTEXT

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

Controller quorum (same on all nodes)

controller.listener.names=CONTROLLER

controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093

Log directory for this broker

log.dirs=/tmp/kraft-broker-1-logs

2. Open config/kraft/broker-2.properties:

```
process.roles=broker
node.id=5

listeners=PLAINTEXT://:9094
advertised.listeners=PLAINTEXT://localhost:9094

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

controller.listener.names=CONTROLLER
controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093

log.dirs=/tmp/kraft-broker-2-logs
```

3. Open config/kraft/broker-3.properties:

```
process.roles=broker
node.id=6

listeners=PLAINTEXT://:9096
advertised.listeners=PLAINTEXT://localhost:9096

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

controller.listener.names=CONTROLLER
controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093

log.dirs=/tmp/kraft-broker-3-logs
```

Notes:

- Brokers use process.roles=broker and must not include their node.id in the controller.quorum.voters list when they are broker-only nodes.kafka.apache+1
- Each broker has its own listener port (9092, 9094, 9096) and its own log.dirs, allowing multiple brokers on the same machine.michael-noll+2

Step 12: Generate Cluster ID and Format All Nodes

You must format storage for each controller and broker with the **same** cluster ID, once per log.dirs directory.docs.confluent+1

1. Generate a cluster ID (once):

```
cd $KAFKA_HOME
CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
echo "Cluster ID: $CLUSTER_ID"
```

2. Format each controller's log directory:

```
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/controller-1.properties
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/controller-2.properties
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/controller-3.properties
```

3. Format each broker's log directory:

```
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/broker-1.properties
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/broker-2.properties
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/broker-3.properties
```

If you change any of the log.dirs or KRaft metadata settings later, delete the corresponding log directories and re-run the format commands with the same CLUSTER_ID or a new one.kafka.apache+1

Step 13: Start the Controllers, Then the Brokers

1. Start the three controllers, each in its own terminal:

```
# Terminal 1
cd $KAFKA_HOME
bin/kafka-server-start.sh config/kraft/controller-1.properties

# Terminal 2
cd $KAFKA_HOME
bin/kafka-server-start.sh config/kraft/controller-2.properties

# Terminal 3
cd $KAFKA_HOME
bin/kafka-server-start.sh config/kraft/controller-3.properties
```

Watch the logs to see one controller become the active controller and the others become followers; the metadata quorum should report as ready.kafka.apache+1

2. Start the three brokers, each in its own terminal:

```
# Terminal 4
cd $KAFKA_HOME
bin/kafka-server-start.sh config/kraft/broker-1.properties

# Terminal 5
cd $KAFKA_HOME
```

```
bin/kafka-server-start.sh config/kraft/broker-2.properties

# Terminal 6
cd $KAFKA_HOME
bin/kafka-server-start.sh config/kraft/broker-3.properties
```

Each broker should connect to the controller quorum, register itself, and report as part of the cluster in the logs.developer.confluent+2

(Optional) You can inspect quorum status with:

```
bin/kafka-metadata-quorum.sh describe --bootstrap-server localhost:9092
```

Step 14: Create a Replicated Topic and Verify

1. Create a topic with replication factor 3 and a few partitions (any broker can be used as --bootstrap-server):

```
cd $KAFKA_HOME
bin/kafka-topics.sh \
  --create \
  --topic kraft-multinode-test \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 3
```

2. List topics to confirm it exists:

```
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

3. Describe the topic to see all three replicas per partition on different brokers:

```
bin/kafka-topics.sh \
  --describe \
  --topic kraft-multinode-test \
  --bootstrap-server localhost:9092
```

The output should show each partition with ReplicationFactor: 3 and preferred/ISR replicas spread across broker IDs 4, 5, and 6.

4. (Optional) Reuse your existing producer/consumer commands from Step 7, pointing to any of the broker ports (9092, 9094, or 9096) as the bootstrap server to validate end-to-end messaging in the multi-broker cluster.

Step 15: Stop and Clean Up the Multi-Node Cluster

1. Stop the producer and consumer (if running) with Ctrl+C.
2. Stop each broker process with Ctrl+C in its terminal.
3. Stop each controller process with Ctrl+C in its terminal.
4. To completely reset this multi-node environment, delete all controller and broker log directories:

```
rm -rf /tmp/kraft-controller-1-logs \
    /tmp/kraft-controller-2-logs \
    /tmp/kraft-controller-3-logs \
    /tmp/kraft-broker-1-logs \
    /tmp/kraft-broker-2-logs \
    /tmp/kraft-broker-3-logs
```

You will need to re-run the kafka-storage.sh format commands with a new or existing CLUSTER_ID before starting this multi-controller, multi-broker cluster again.kafka.apache+1